

PROGRAMAÇÃO WEB

AULAS TEÓRICAS

ANO LECTIVO DE 2024/25

1° SEMESTRE



Capítulo 3

Manipulação de Dados em .Net Core 8

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB



3.1. Entity Franmework Core 8

Programação Web – 2024/2025 – LEI D+GE+PL – ISEC/IPC - JB



Entity Framework

.NET Core

Core

ENTITY FRAMEWORK CORE 8



- **O que é que é a Entity Framework (EF) ?**
- A **Entity Framework (EF)** é um **ORM**, ***object-relacional mapper***, que permite aos programadores .NET trabalhar com dados relacionais utilizando objetos específicos do domínio.
- Elimina a necessidade da maior parte do código de acesso a dados que geralmente se tinha de criar.
- Tem como **finalidade** abstrair os vínculos com uma base de dados relacional.

Programação Web - 2024/2025 – LEI-D+CET – ISEC/IPC - JB

ENTITY FRAMEWORK CORE 8



- **Vantagens da utilização da Entity Framework?**
- O Entity Framework é um ORM, e os ORMs têm como objetivo aumentar a produtividade, reduzindo a tarefa redundante de persistir os dados utilizados nas aplicações.
- Recursos do Entity Framework
  - Entity Framework é uma tecnologia de mapeamento objeto-relacional (ORM) leve e extensível.
  - O Entity Framework suporta diversas plataformas, como o Windows, Linux e macOS.

Programação Web 2024/2025 – LEI+D+CET+I – ISEC/ARC - JB

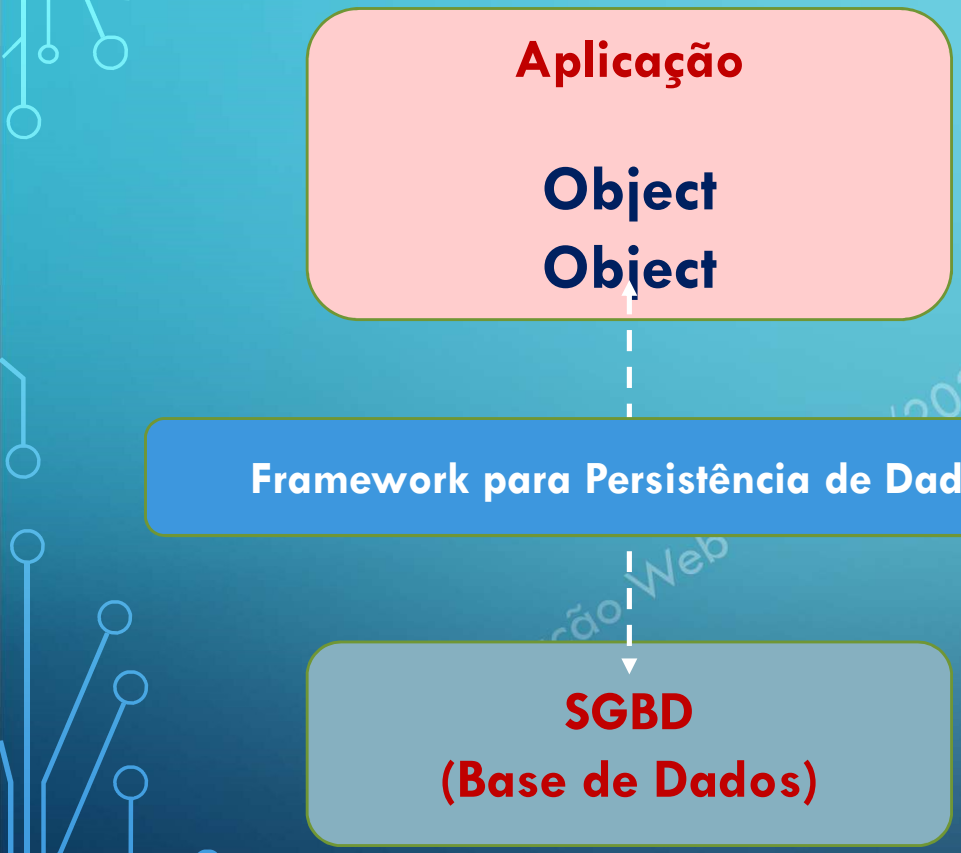
ENTITY FRAMEWORK CORE 8



- O Entity Framework suporta fontes de dados relacionais e não relacionais.
  - O Entity Framework funciona de forma eficiente com bases de dados amplamente utilizadas, como o SQL Server, SQL Server Compact, SQLite e PostgreSQL.
- O Entity Framework facilita a execução de operações de criação, leitura, atualização e eliminação (CRUD), suportando bases de dados.
  - Também torna mais fácil a realização de testes unitários, mantendo tabelas em memória.

Programação Web - 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

ENTITY FRAMEWORK CORE 8 (EF)

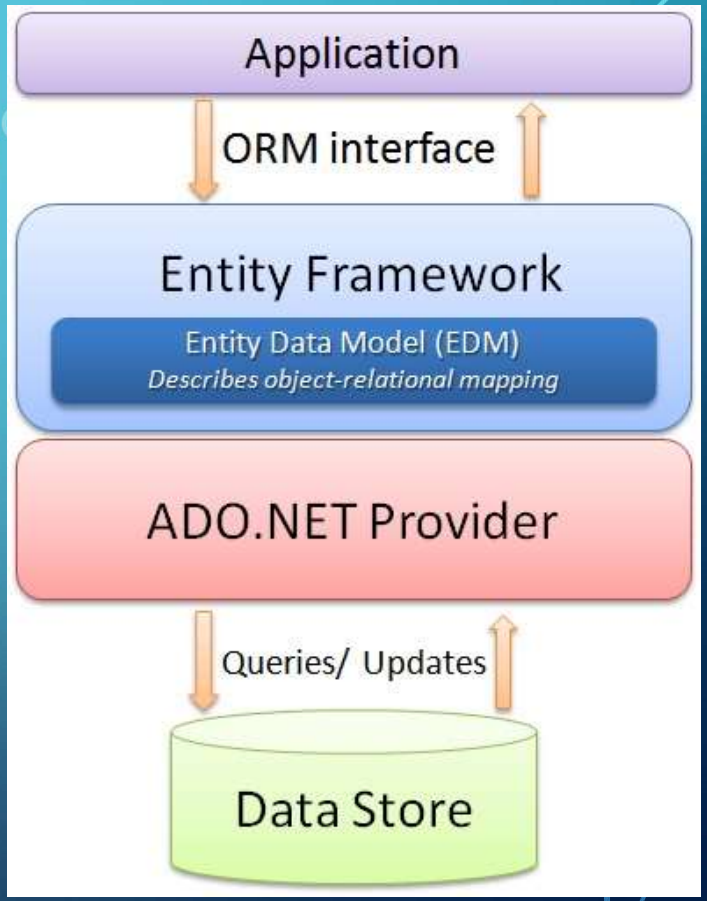


Entity Framework Core

2025 – LEI D+CE+PL – ISEC/IPC - JB

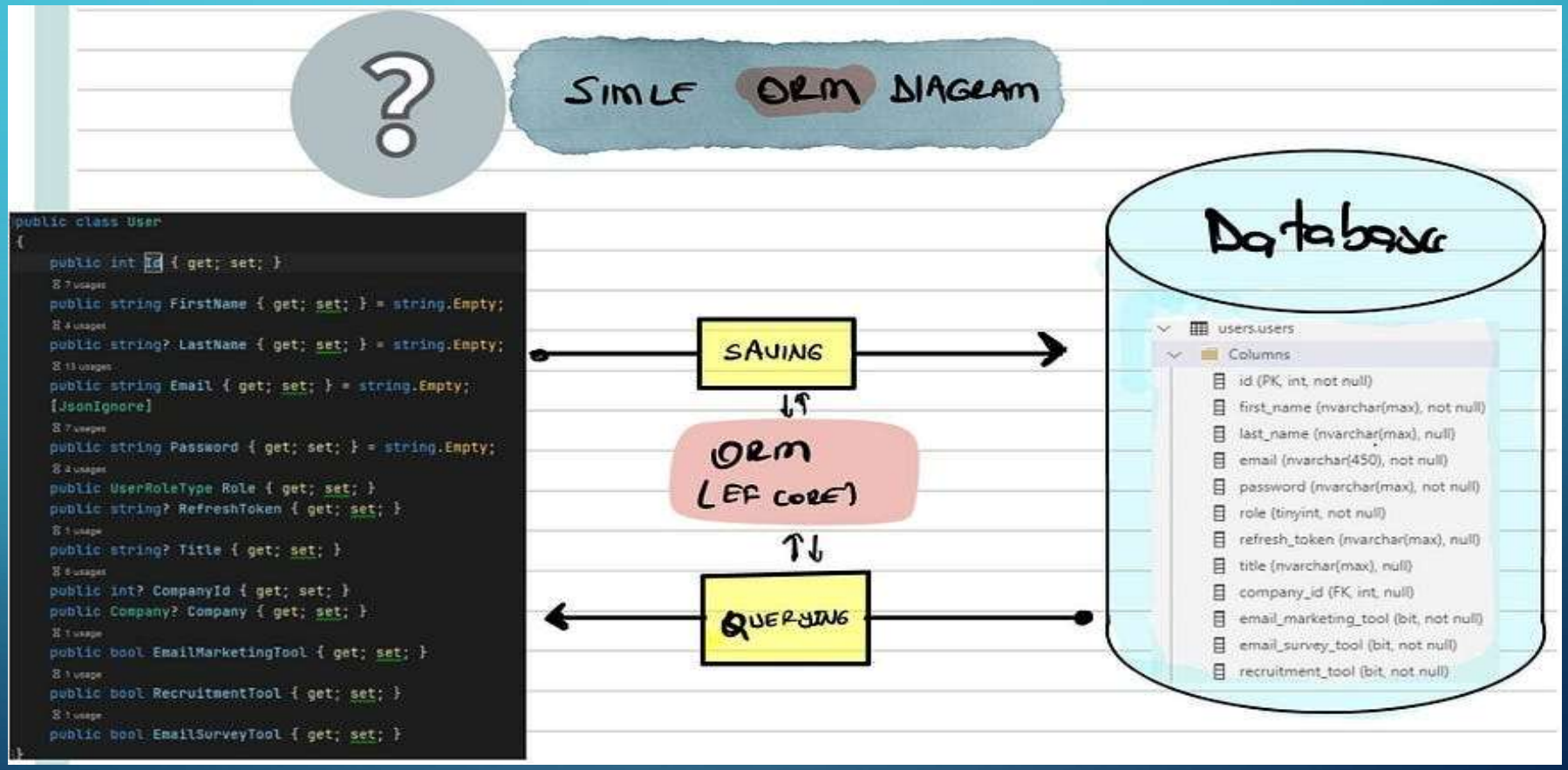
ENTITY FRAMEWORK CORE 8 (EF)

- *Object Relational Mapper (ORM)*
- *Open Source*
- Aumentar a produtividade do programador, reduzindo tarefas redundantes para persistência de dados



Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IP

ENTITY FRAMEWORK CORE 8 (EF)



ENTITY FRAMEWORK CORE 8

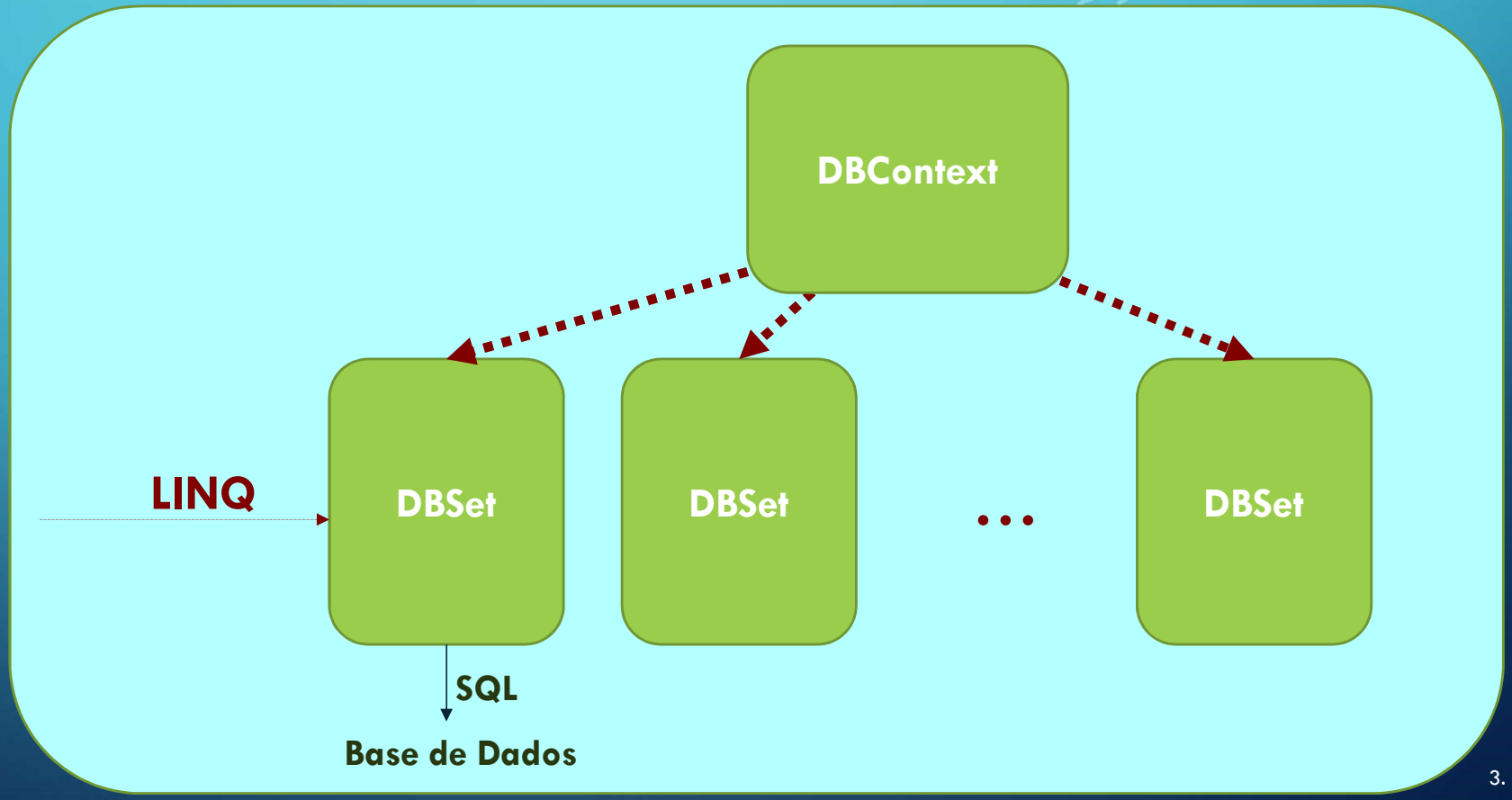


Existem três abordagens possíveis de serem usadas:

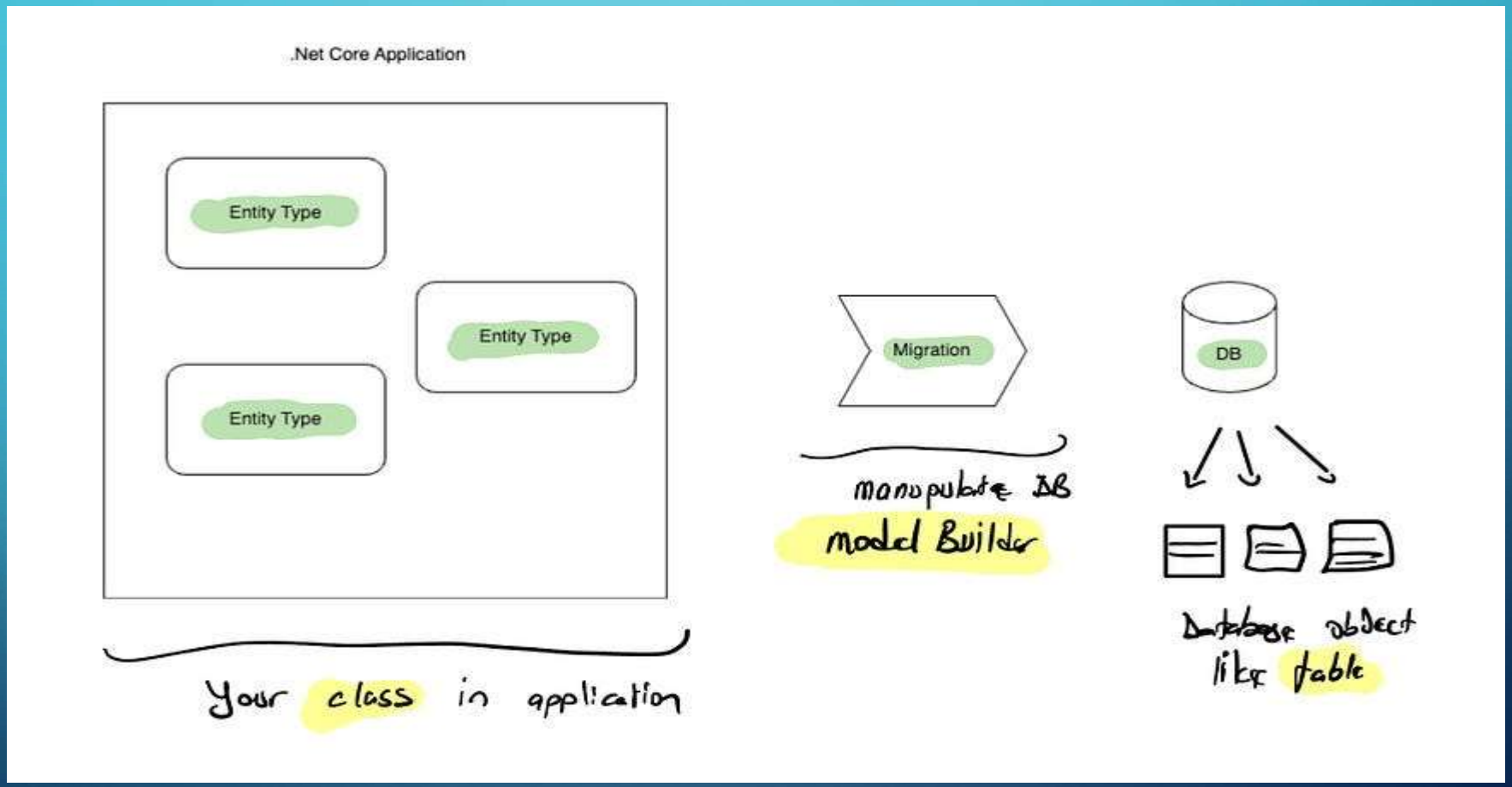
- **Code First**
  - A abordagem Code First permite-nos criar um modelo e uma relação utilizando classes e, em seguida, criar a base de dados a partir dessas classes.
    - Permite-nos trabalhar com o Entity Framework de uma forma orientada a objetos.
  - Iremos dar particular atenção à utilização de *Code First*

Programação Web - 2024/2025 - EFD+CF-PL – ISEC/IPC - JB

1. Code First



ENTITY FRAMEWORK CORE 8 (EF)



ENTITY FRAMEWORK CORE 8



- **Data Base First**
  - A abordagem Database First permite-nos criar um modelo de entidades a partir da base de dados existente.
    - Esta abordagem ajuda-nos a reduzir a quantidade de código que precisamos de escrever.
    - O procedimento seguinte irá criar um modelo de entidade utilizando a abordagem Database First.

Programação Web 2024/2025 – LEI DTCE+PL – ISEC/IPC - JB

ENTITY FRAMEWORK CORE 8



- **Model First**

- Nesta abordagem, as classes do modelo e as suas relações são criadas primeiro usando o designer ORM, e a base de dados física será gerada usando este modelo.
  - A abordagem Model First significa que criamos um diagrama da entidade e da relação que será automaticamente convertido num modelo de código.
    - Model First - Esta abordagem era possível com versões anteriores do ASP.Net, se bem que já na altura pouco suportada pela MS mas actualmente praticamente já não é usada

Programação Web - 2024/2025 - LEI DTCF+PL - ISEC/IPC - JB

ENTITY FRAMEWORK CORE 8



- **Em resumo:**

1. *Code First*

- Parte-se das classes do domínio e a ***EF*** gera a Base de Dados

2. *Database First*

- Parte-se da Base de Dados para obter as classes de domínio

Programação Web – 2024/2025 – TSI D+CE+PL – ISEC/IPC - JB

ENTITY FRAMEWORK – CODE FIRST



Etapas a seguir para a utilização da ***Entity Frameworke em CodeFirst***

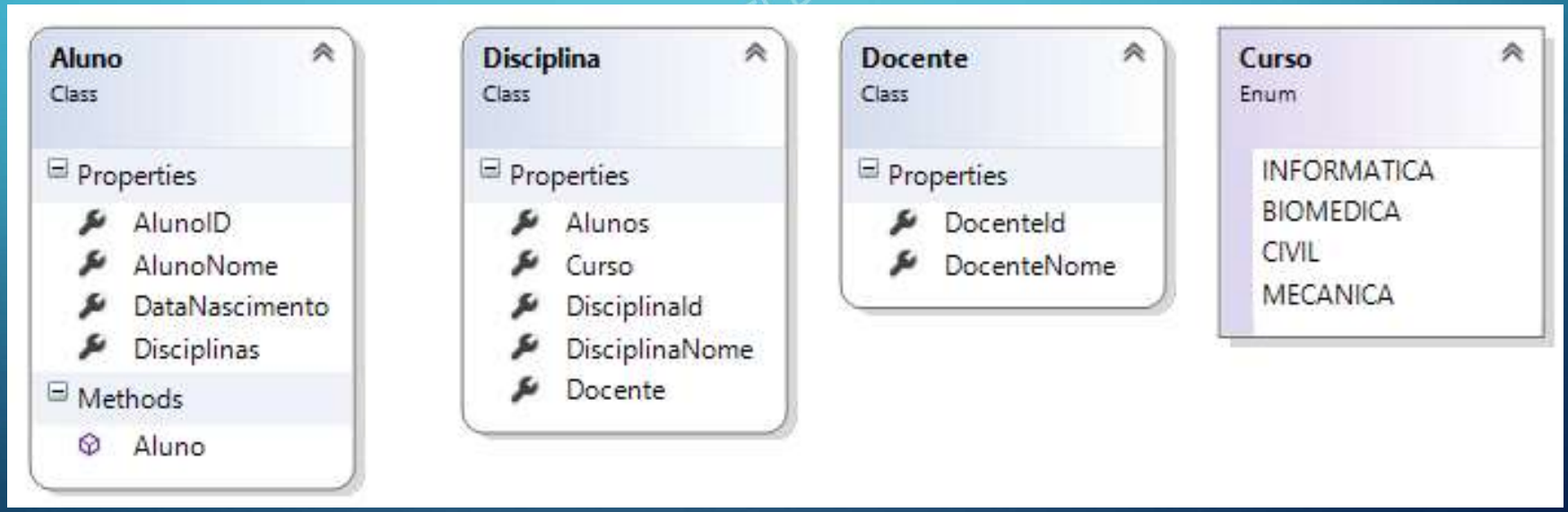
1. Instalação da ***Entity Framework***
2. *Definição do* ***Modelo de Domínio***
3. Criação do ***Contexto*** (classe que herda da ***DbContext*** e que é uma abstração da base de dados usada no acesso aos dados
4. Registar no método ***Services*** de ***Program.cs*** o ***DBContext*** como um serviço
5. Especificação da ***Connection String*** no ficheiro ***appsettings.json***

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC

ESPECIFICAÇÃO DAS ENTIDADES A UTILIZAR

2. *Definição do **Modelo de Domínio***

Na pasta *Models* crie as classes que representem as entidade de domínio
Por exemplo:



CRIAÇÃO DO MODELO CURSO



```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace AutoWEB.Models
{
    26 referências
    public class Curso
    {
        15 referências
        public int Id { get; set; }
        [Display(Name = "Nome", Prompt = "Introduza o Nome do curso!")]
        [Required(ErrorMessage = "Indique o nome do curso!")]
        19 referências
        public string Nome { get; set; }
        [Display(Name = "Descrição", Prompt = "Introduza a Descrição do curso!")]
        [Required(ErrorMessage = "Descreva o curso!")]
        11 referências
        public string Descricao { get; set; }
        [Display(Name = "Descrição Resumida", Prompt = "Introduza a Descrição Resumida do curso!")]
        [Required(ErrorMessage = "Descreva resumidamente o curso!")]
        17 referências
        public string DescricaoResumida { get; set; }
        [Display(Name = "Requisitos", Prompt = "Introduza os Requisitos do curso!")]
        [Required(ErrorMessage = "Indique os requisitos do curso!")]
        10 referências
        public string Requisitos { get; set; }
        [Display(Name = "Idade mínima", Prompt = "Idade Mínima",
            Description = "Idade mínima para poder frequentar esta formação")]
        [Required(ErrorMessage = "Indique a Idade Mínima para este curso!")]
        [Range(14, 100, ErrorMessage = "Mínimo: 14 anos, Máximo: 100 anos")]
        10 referências
        public int IdadeMinima { get; set; }
        [Display(Name = "Curso activo?")]
        12 referências
        public bool Disponivel { get; set; }
        [Display(Name = "Curso em Destaque?")]
        11 referências
        public bool EmDestaque { get; set; }
        [Required(ErrorMessage = "Indique o Preço do curso!")]
        [Display(Name = "Preço", Prompt = "Introduza o Preço", Description = "Preço do curso")]
        [DisplayFormat(DataFormatString = "{0:C2}")]
        12 referências
        public decimal? Preco { get; set; }
        [Display(Name = "Categoria")]
        11 referências
        public int? CategoriaId { get; set; }
        10 referências
        public Categoria categoria { get; set; }
    }
}
```

3. Definição do **Contexto** da Base de Dados

Na pasta ***Data*** crie o arquivo ***ApplicationDBContext***

```
using AutoWEB.Models;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace AutoWEB.Data
{
    16 referências
    public class ApplicationDbContext : IdentityDbContext <ApplicationUser>
    {
        19 referências
        public DbSet<Curso> Cursos { get; set; }
        17 referências
        public DbSet<Categoria> Categorias { get; set; }
        12 referências
        public DbSet<Agendamento> Agendamentos { get; set; }
        18 referências
        public DbSet<TipoDeAula> TipoDeAula { get; set; }

        0 referências
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }
    }
}
```

DBCONTEXT COMO SERVIÇO – PROGRAM.CS

4. Registar em ***Program.cs*** o ***DBContext*** como um serviço

No ficheiro ***program.cs***, tem de se especificar o ***DBContext*** como um serviço

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));
...
```

Programação Web – 2024/2025 – LEI DI-CE+PL – ISEC/IPC – JB

ESPECIFICAÇÃO DA *CONNECTION STRING*

5. Especificação da ***Connection String*** no ficheiro ***appsettings.json***

No ficheiro *appsettings.json*, define-se uma *string* de conexão com a base de dados **usando uma instância local da BD**

```
"ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=FrutariaWeb;Trusted_Connection=True;
MultipleActiveResultSets=true"
 },
 ...
```

Programação Web – 2024/2025 – LEI DI-ESTG+PL – ISEC/IPC – JB



Entity Framework Core 8

Data Base First

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/DEIS – JB

ENTITY FRAMEWORK – DATA BASE FIRST

O ***Entity Framework Database First*** é uma abordagem alternativa no desenvolvimento de sites para a *Web* utilizando-se o ***Entity Framework Core 8***

- Pode ser utilizada quer se disponha já de uma base de dados, por exemplo a utilizada num site que se quer actualizar ou quer se tenha desenvolvido especifica e previamente uma Base de Dados para um novo site
  - No entanto é comum que no desenvolvimento de raíz de um site, se utilize a abordagem *Code First*

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC

ENTITY FRAMEWORK – DATA BASE FIRST

No caso de se dispor já de uma Base de Dados do um *site* em produção, a Base de Dados respectiva já deverá ter um número muito grande de tabelas as quais também já deverão conter um número muito significativo de dados inseridos

- Neste caso não faria sentido não se utilizar essa Base de Dados, por exemplo numa actualização desse *site*

Programação Web – 2024/2025 – EI D+T+T+PL – ISEC/IPC - JB

ENTITY FRAMEWORK – DATA BASE FIRST

Nesta situação de se dispor já de uma Base de Dados, se se adoptasse a abordagem *Code First*, poderiam ocorrer alguns dificuldades:

- Ter-se-ia de se definir manualmente cada uma das classes C# correspondentes a cada uma das tabelas e as respectivas relações
- Não se poderia usar migrações *Code First* porque a base de dados existente provavelmente estará a ser gerida de uma maneira diferente.
  - Embora isto não fosse um problema, perder-se-ia uma das grandes vantagens de se usar o *Code First*

Programação Web 2024/2025 – LEI DI CE+PL – ISEC/DEC - ...

ENTITY FRAMEWORK – DATA BASE FIRST

Deste modo, nestas situações é preferível usar a abordagem ***Data Base First***:

- O que se faria criando manualmente o código das classes ***C#***, será feita pela ***EF*** através da inspecção da base de dados existente.
- O resultado dessa acção de análise da *BD* pela ***EF*** será então a criação das classes de dados ***C#***

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/ISEC – JB

ENTITY FRAMEWORK – DATA BASE FIRST

Para que o ***EF*** realize essa inspecção da *BD* e crie as respectivas classes ***C#***, basta executar o seguinte comando:

Scaffold-DbContext "MyConnectionStringHere“

E o Entity Framework fará o resto por nós

Programação Web – 2024/2025 – LEI D+CE+RL – ESEC/IPC - JB



3.2. Migrations

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

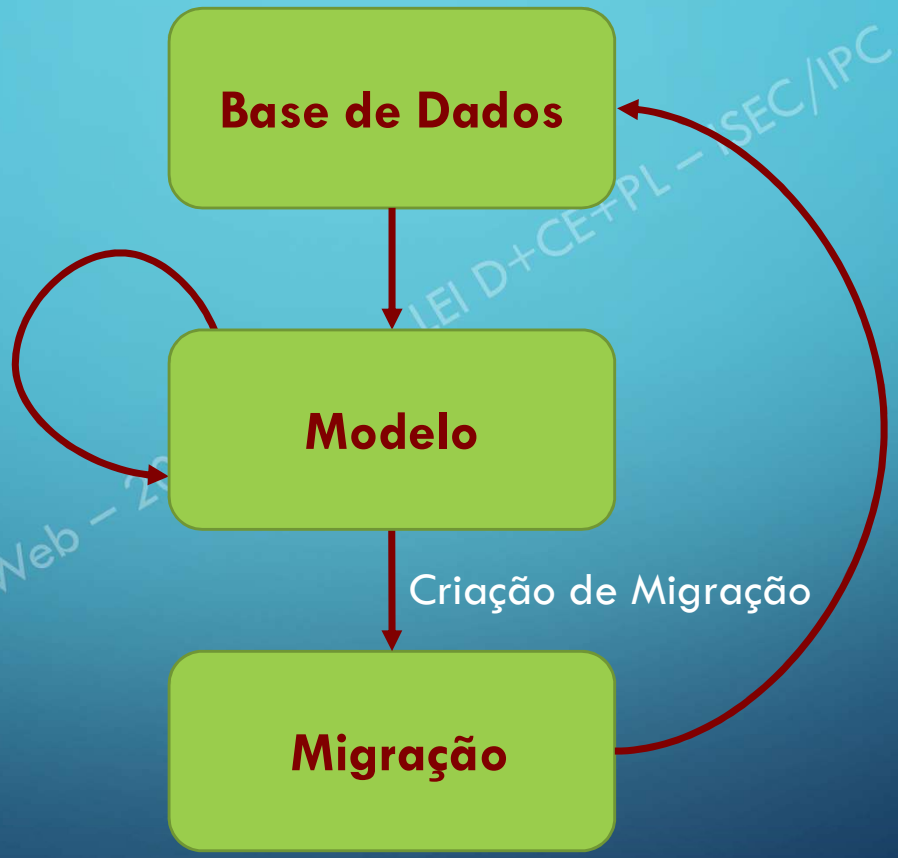
EF CORE MIGRATIONS



A *EF Core API* constrói o modelo *EF Core* a partir das classes de domínio (entidades) e as *migrations EF Core* criarão ou atualizarão o esquema da base de dados com base no modelo *EF Core*

Sempre que as classes de domínio são alteradas, é necessário executar novamente a *migration* para manter o esquema da base de dados atualizado

PROCESSO ITERATIVO



Alterações ao Modelo

Executa

Manter a Base de Dados atualizada

Permite obter o histórico da Base de Dados
Os scripts gerados na Migrations permitem fazer o Upgrade e o Downgrade das alterações

Programação Web – 2 LEI D+CE+PL – SEC/IPC - JB

EF CORE MIGRATIONS

As *migrations EF Core* são um conjunto de comandos que se executam na consola do gestor de pacotes *NuGet* ou na interface de linha de comando dotnet

A tabela seguinte mostram-se os comandos de *migration* mais importantes no *EF Core*

Programação Web 2024/2025 – LEI D+CE+RL – ISEC/IPC - JB



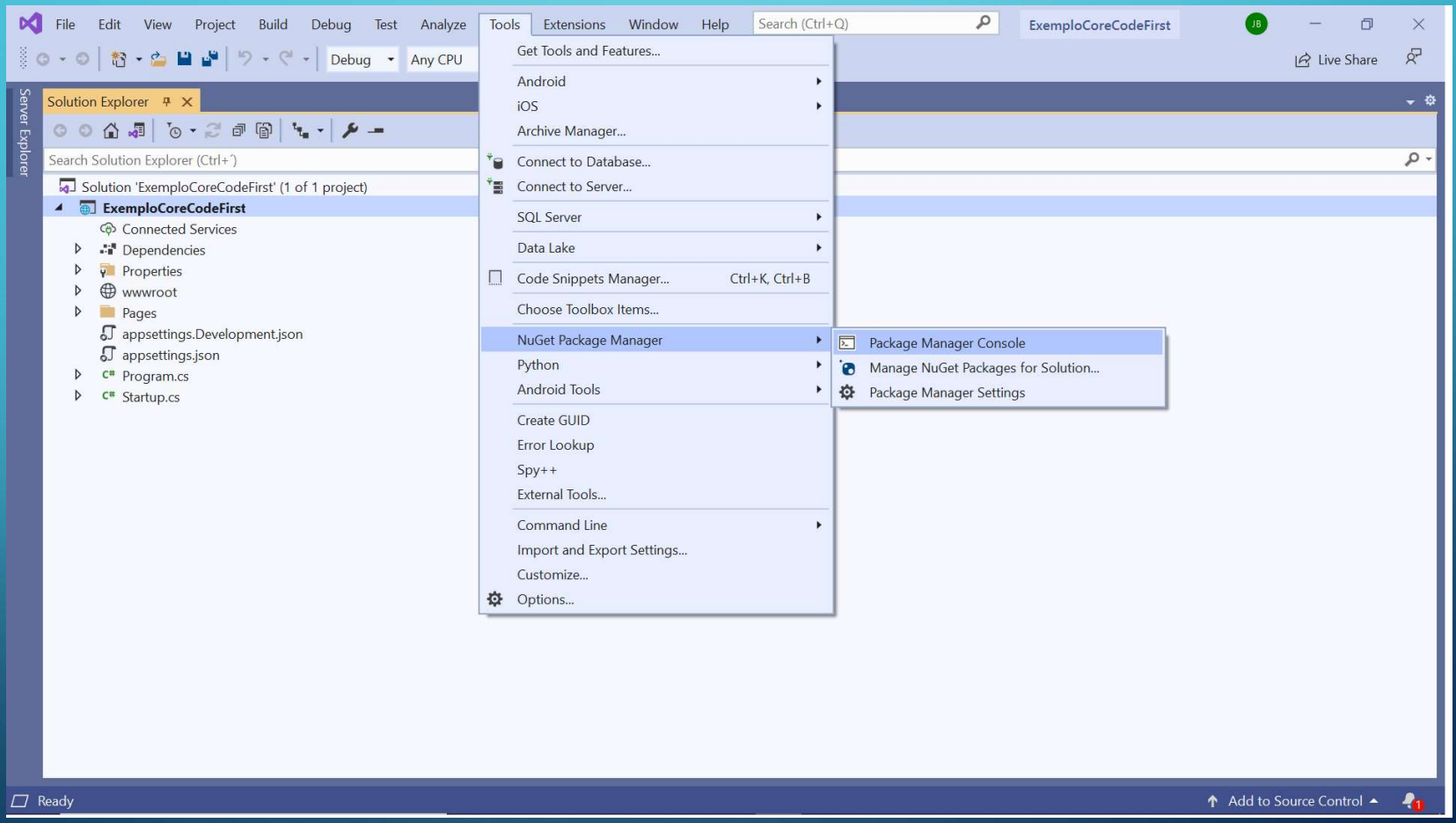
EF CORE MIGRATIONS - COMANDOS

Comando PMC	Comando dotnet CLI	Utilização
add-migration <migration name>	Add <migration name>	Cria a migration adicionando uma migration snapshot.
Remove-migration	Remove	Remove a última migration snapshot.
Update-database	Update	Actualiza o esquema da base de dados baseada na útlima migration snapshot.
Script-migration	Script	Gera um script SQL usando todas as migration snapshots.

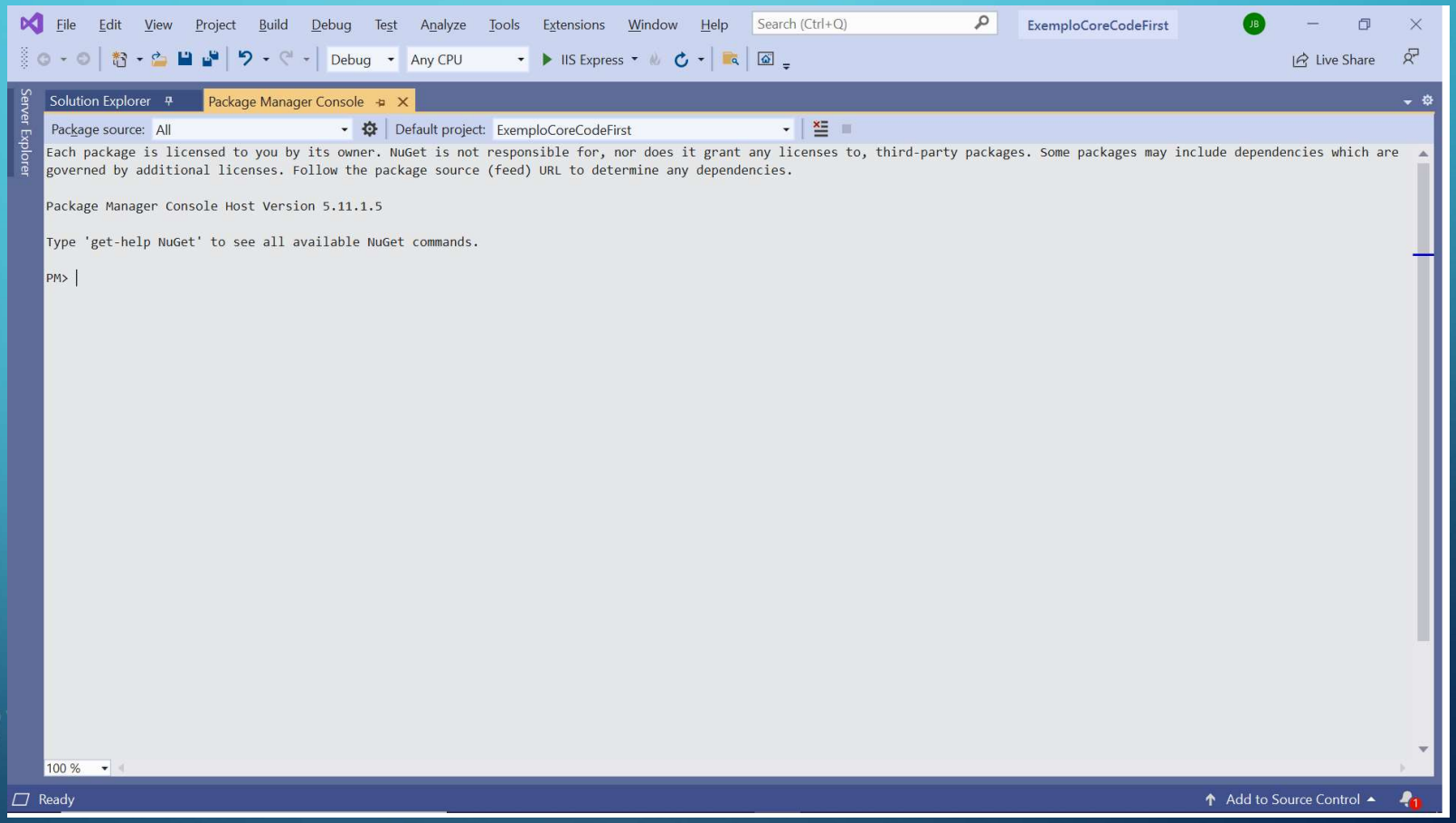
EF CORE MIGRATIONS – MAIS ALGUNS COMANDOS

Package Manager Console	Description
add-migration [name]	Create a new migration with the specific migration name.
remove-migration	Remove the latest migration.
update-database	Update the database to the latest migration.
update-database [name]	Update the database to a specific migration name point.
get-migrations	Lists all available migrations.
script-migration	Generates an SQL script for all migrations.
drop-database	Drop the database.

ATIVAÇÃO DAS *MIGRATIONS*

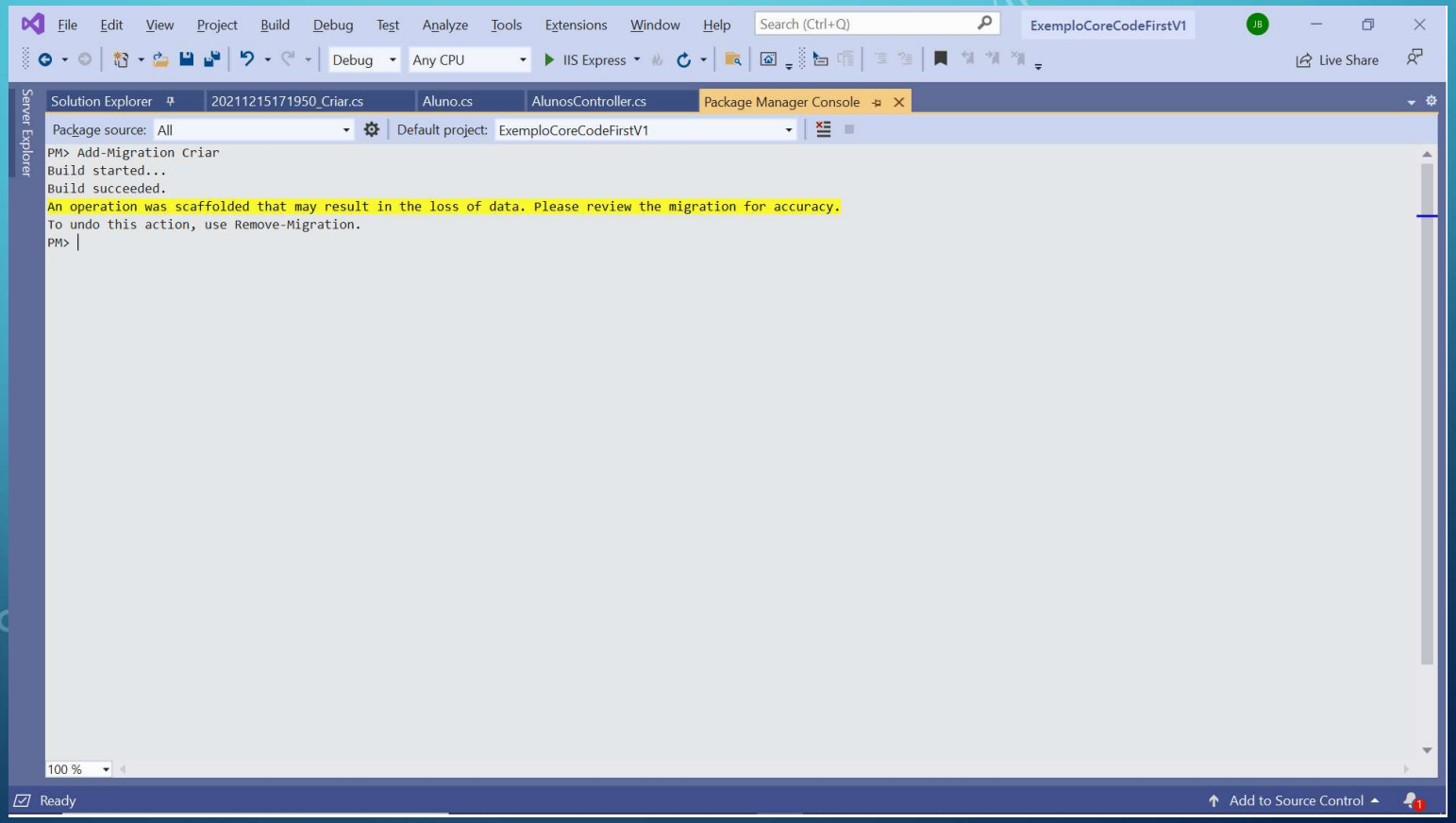


PACKAGE MANAGER CONSOLE



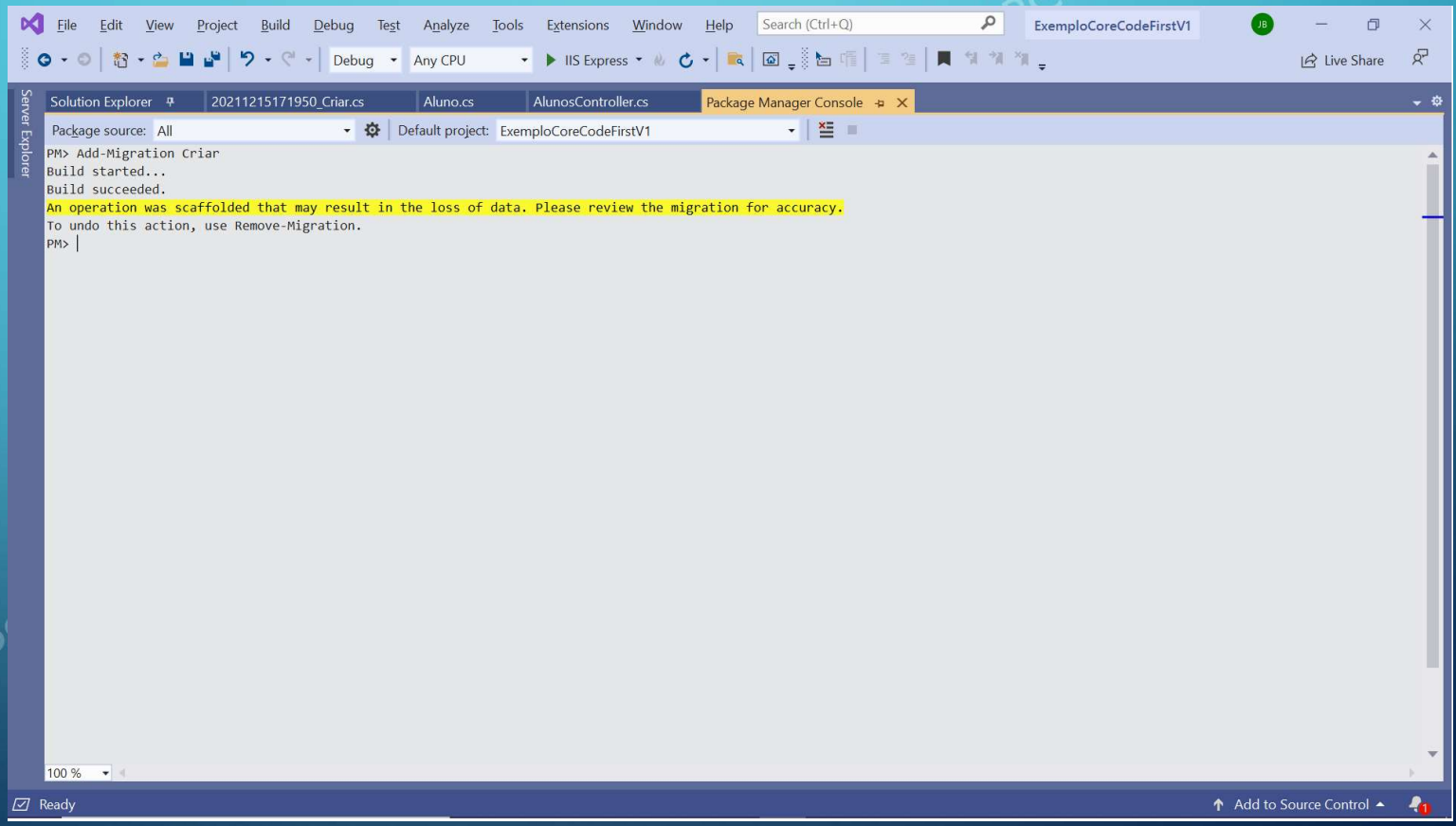
MIGRATIONS

PM> Add-Migration Criar ou > `dotnet ef migrations add Criar`



MIGRATIONS

PM> update-database





Mas qual o objectivo das Migrações (Migrations) ?

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/IPC - JB

EF CORE MIGRATIONS - COMANDOS



Adicionando uma Migration

Inicialmente, definem-se as classes de domínio

Após esta definição, ainda não se tem a base de dados para armazenar os dados das classes de domínio.

O passo seguinte é então criar essa BD e para isso utiliza-se uma *migration*

Selecciona-se a opção do menu Ferramentas -> Gerenciador de pacotes NuGet -> Console do gerenciador de pacotes no Visual Studio

Com isto é aberto a Consola do NuGet e nesta consola executa-se o comando para adicionar uma migration

PM> add-migration MyFirstMigration

Caso se use a interface de linha de comando dotnet, executa-se o comando ***CLI***

> dotnet ef migrations add MyFirstMigration

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/PRC - JB

PM> Update-Database

EF CORE MIGRATIONS - COMANDOS

Criar ou Actualizar a Base de Dados

Use um dos seguintes comandos para criar ou atualizar o esquema da base de dados

Package Manager Console

PM> Update-Database

CLI

> dotnet ef database update

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

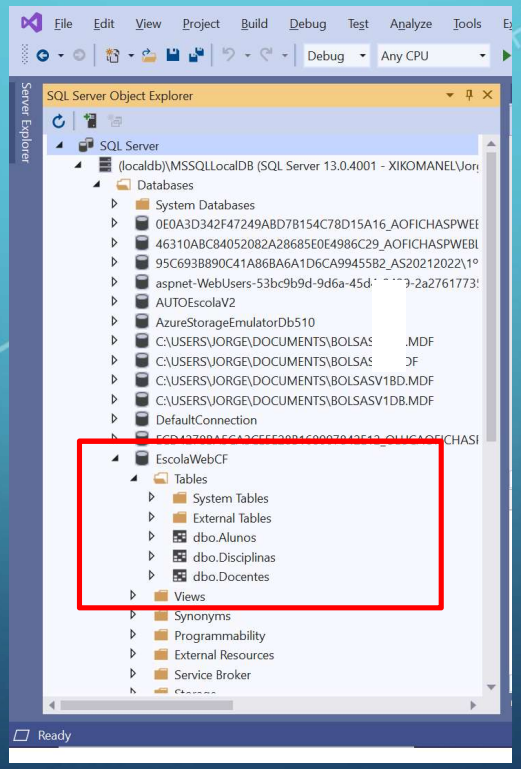
O comando *Update-Database* criará a base de dados com base no contexto e nas classes de domínio e no *snapshot* da *migration*, criado usando o comando *add-migration*

- Na primeira migração, também será criada uma tabela chamada *__EFMigrationsHistory*, que armazenará o nome de todas as *migrations*, como e quando serão aplicadas à base de dados

EF CORE MIGRATIONS - COMANDOS

Criar ou Actualizar a Base de Dados

Após a execução deste comandos em que ***MyFirstMigration*** é o nome de uma ***migration*** será criada no SQL Server a Base de Dados e as respectivas tabelas



Programação Web - PL – ISEC/IPC - JB

MIGRATTIONS – PASSOS PRINCIPAIS

Usar Migrações Code First

2 Passos Principais:

1. **Adicionar as alterações**

PM> Add-Migration *darumnomequalqueraadicao*

2. **Actualizar a Base de Dados**

PM> Update-Database

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB



MAPEAMENTO ENTRE TIPOS C# E TIPOS SQL

Tipo de Dados em C#	Tipo de Dados na coluna da DB	Tipo de dados da Coluna da chave Primária e Comprimento
int	int	int, Coluna Identity
string	nvarchar(Max)	nvarchar(128)
decimal	decimal(18,2)	decimal(18,2)
float	real	real
byte[]	varbinary(Max)	varbinary(128)
datetime	datetime	datetime
bool	bit	bit
byte	tinyint	tinyint
short	smallint	smallint
long	bigint	bigint
double	float	float
char	No mapping	No mapping
sbyte	No mapping (throws exception)	No mapping
object	No mapping	No mapping

DATA ANNOTATIONS



- As ***Data Annotations*** fornecem uma forma simples e poderosa de personalizar os modelos EF Core diretamente nas classes de entidades.
  - Ao adicionar alguns atributos, pode-se ajustar a forma como o EF Core lida com os dados, desde a definição de chaves primárias e relações até à definição de regras de validação e opções de formatação.
  - Esta abordagem oferece um meio-termo entre a facilidade das convenções e a flexibilidade da API Fluent, tornando-a uma escolha adequada em muitas situações

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/MF – JB



Data Annotations

- No Entity Framework Core (EF Core), as ***Data Annotations*** fornecem uma forma de configurar o modelo de dados, adicionando atributos às classes de entidades e às suas propriedades.
  - As data annotations oferecem uma forma direta de especificar regras e comportamentos diretamente no código, tornando muitas vezes o código mais legível e de fácil manutenção.

Programação Web – 2024/2025 – LEI D+CL PL – ISEC/IPC - JB

DATA ANNOTATIONS



- ***Data Annotations***

  - Simples de especificar
  - Menos flexível
  - Útil para anotações simples como obrigatórios, tamanhos máximos…
  - *DataAnnotations* tem limitações de configuração

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

DATA ANNOTATIONS: KEY

```
using System.ComponentModel.DataAnnotations;
```

- Chaves Primárias

```
[Key]
private int NAluno { get; set; }

[Key]
[DatabaseGenerated(DatabaseGeneratedOption.None)]
private string SIGLA { get; set; }
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

DATA ANNOTATIONS: KEYS COMPOSTAS

- Chaves Primárias Compostas

```
public class Inscricoes
{
    [Key]
    [Column(Order=1)]
  public int DisciplinaId { get; set; }

    [Key,Column(Order=2)]
    public int AlunoId { get; set; }

  public DateTime DataInscricao { get; set; }
}
```

DATAANNOTATIONS: NOTMAPPED

```
[NotMapped]
public int Idade { get; set; }

public string PrimeiroNome
{
   get { return AlunoNome; }
}
public int IdadeGet
{
     get { return 0; }
}
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

DATAANNOTATIONS: ÍNDICES

```
[Index]
public int DisciplinaId { get; set; }
```

```
[Index(IsUnique = true)]
public int DisciplinaId { get; set; }
```

Programação Web - 2024/2025 – LEI D+CL+P – ISEC/IPC - JB



3.3. LINQ

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

O que é o LInQ?

O Language Integrated Query (LINQ) é uma das funcionalidades mais poderosas do C#.

- Permite a escrita de consultas diretamente na linguagem C# para manipular e consultar dados de diferentes fontes de dados, como coleções, bases de dados, documentos XML e muito mais.
- O LINQ facilita o trabalho com dados, proporcionando uma forma consistente e legível de consultar e transformar dados.

Programação Web – 2024/2025 – LEI D+CE+TC – ISEC/IPC - JB

Vantagens do LINQ?

- Sintaxe Unificada: o LINQ fornece uma sintaxe uniforme para consultar diferentes fontes de dados (por exemplo, arrays, listas, XML, bases de dados).
- Legibilidade: as consultas LINQ são geralmente mais legíveis e concisas do que os ciclos equivalentes ou outras construções de código.

Programação Web - 2024/2025 - UFCD+C#+PL - ISEC/IPC - JB



- **Fortemente tipado:** o LINQ está integrado no C#, o que significa que obtém verificação em tempo de compilação e suporte para o IntelliSense.

- **Abordagem declarativa:** o LINQ permite descrever o que se pretende fazer com os dados, em vez de como o fazer, resultando num código mais claro e de fácil manutenção.

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/DEC - JB



Conceitos Básicos de LINQ

1. Fonte de dados: a coleção ou sequência de dados que se pretende consultar. Pode ser um array, uma lista, uma tabela de base de dados, etc.

2.º Consulta: o conjunto de operações que se define para se obter os dados necessários da fonte de dados.

3. Execução: Quando a consulta é executada, extrai-se os dados da fonte de dados com base nas condições da consulta.

Programação Web – 2024/2025 – LEI D+CE+AI – ISEC/IPC - JB



Sintaxe LINQ

O LINQ disponibiliza dois tipos de sintaxe:

Query Sintax: é semelhante ao SQL e é mais legível para quem está familiarizado com o SQL.

Exemplo:

```
var numerosPares = from num in numbers
                   where num % 2 == 0
                   select num;
```

Programação Web – 2024/2025 – LEI+CE+PL – ISEC/IPC - JB



Method Syntax: é mais compacto e frequentemente utilizado no encadeamento de métodos.

```
var números pares = números.Where(num => num % 2 == 0);
```

Ambas as sintaxes são equivalentes e pode utilizar a que achar mais legível mas o Method Syntax tem mais operadores e é esta sintaxe que está em evolução ou seja o que é novo só aparece nesta versão

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JP

Principais operadores do LINQ

O LINQ fornece um vasto conjunto de operadores para manipular dados.

Alguns dos mais utilizados são:

Where: Filtra elementos com base numa condição.

```
var números pares = números.Where(num => num % 2 == 0);
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB

Select: projeta cada elemento de uma sequência numa nova forma.

```
var quadrados = números. Select(num => num * num);
```

OrderBy/OrderByDescendente: Classifica os elementos por ordem crescente ou decrescente.

```
var sortedNumbers = numbers. OrderBy(num => num);
```

Programação Web - 2024/2025 – LEI D - CE+PL – ISEC/IPC - J

GroupBy: agrupa elementos que partilham um atributo comum.

```
var groupedNumbers = numbers.GroupBy(num => num % 2);
```

Join: Une duas sequências com base numa chave.

```
var joinedData = from num in numbers
                 join num2 in otherNumbers on num equals num2
                 select new { num, num2 };
```

Programação Web – 2024/2025 – LEI DI-CE+PL – ISEC/IPC - JS

Aggregate: executa uma operação em cada elemento da sequência e devolve um único resultado.

```
var soma = números. Aggregate((a, b) => a + b);
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JE

Trabalhar com diferentes fontes de dados

O LINQ não está limitado a coleções; pode-se utilizá-lo para consultar várias fontes de dados tais como:

- LINQ to Objects: consulta coleções como arrays, listas, etc.

```
var result = from num in numeros where num > 5 select num;
```

- LINQ to SQL: consulta bases de dados relacionais.

```
var query = from p in context.Produtos where p.Preco > 100 select p;
```

Programação Web – 2024/2025 – LEI D+G+PL – ISEC/IPC - JB



LINQ to XML: consulta documentos XML.

```
var xmlQuery = from x in xmlDoc.Elements("Livro") where
(string)x.Element("Author") == "AuthorName" select x;
```

LINQ to Entities: consulta modelos do Entity Framework.

Programação Web – 2024/2025 – LEI D+CR+PL – ISEC/IPC - JB

Deferred Execution vs. Immediate Execution

Um conceito importante no LINQ é a **Deferred Execution**.

As consultas LINQ não são executadas quando são definidas, mas quando são iteradas (por exemplo, utilizando um ciclo foreach).

- Isto é chamado de Deferred Execution.

```
var query = from num in numbers where num > 5 select num;
// A Query não é executada aqui
foreach (var num in query)
{
    Console.WriteLine(num); // A Query só é executada aqui
}
```

Programação Web – 2024/2025 – LEI DI – CE+PL – SEC/IPC – IR

A **Immediate Execution** ocorre quando são chamados métodos como ToList(), ToArray(), First() ou Count(), o que obriga à execução imediata da consulta.

```
var execucaoImediata = numbers.Where(num => num > 5).ToList(); // Executada imediatamente
```

Programação Web – 2024/2025 – LEI D+CE+PL – ISEC/IPC - JB